MediTrust: Pharmaceutical Supply Chain Verification

Comprehensive Technical Report — Phase 2: Smart Contract Execution & Security Validation

1. Executive Summary

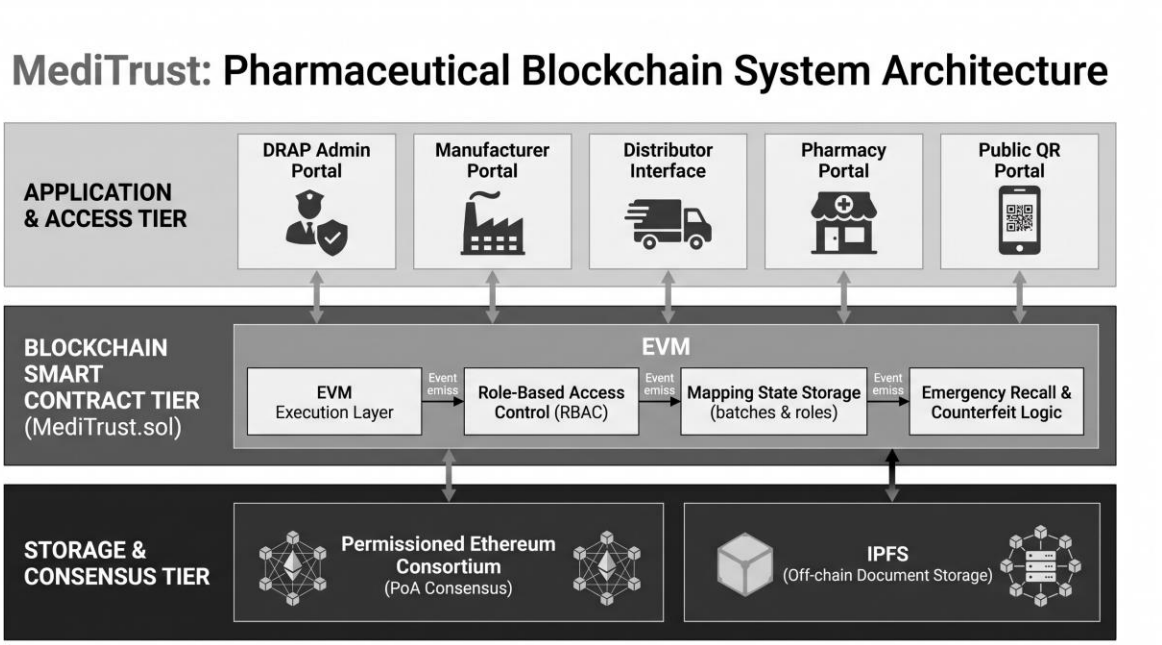
The MediTrust platform is a decentralized solution engineered to mitigate global pharmaceutical counterfeiting, track batch provenance across multi-tiered supply networks, and enforce real-time emergency product recalls. Phase 2 focused on compiling, deploying, and rigorously testing the core Solidity smart contract (MediTrust.sol) within the Remix IDE environment using Solidity compiler 0.8.20. Key objectives included setting up role-based access control (RBAC), verifying custody handoffs, and evaluating on-chain security mechanisms for automated clone detection.

> **Key Objective**
> *Core Validation Milestone: Successfully executed and audited role authorization, automated scan counter incrementing for duplicate detection, and absolute DRAP emergency recall circuit-breaker locks on-chain.*

2. System Architecture & Role Authorization Mapping

Role	Entity Description	Remix VM Test Address	Authorizations & Security Scope
Regulator	DRAP / Government Admin	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4	Assigns network roles; executes global batch recalls.
Manufacturer	Pharmaceutical Producer	0xAb8483F64d9C6d1EcF9b849Ae6E7dE53415D4409	Registers new medicine batches; initiates custody transfers.
Distributor	Wholesale Supply Node	0x4B20993Bc481154053573A44B2E31D560f85f839	Accepts custody from Manufacturer; transfers to Pharmacy.

Pharmacy	Retail Dispenser	0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB	Holds retail batch custody before public verification.



3. Smart Contract Implementation (MediTrust.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MediTrust {
    address public regulator;

    enum Role { None, Manufacturer, Distributor, Pharmacy }

    struct Batch {
        uint256 batchId;
        string medicineName;
        address currentOwner;
        Role stage;
        bool isRecalled;
        uint256 scanCount;
    }

    mapping(uint256 => Batch) public batches;
    mapping(address => Role) public userRoles;

    event BatchRegistered(uint256 indexed batchId, string medicineName, address indexed
manufacturer);
```

```
    event CustodyTransferred(uint256 indexed batchId, address indexed from, address indexed to,
Role stage);
    event BatchRecalled(uint256 indexed batchId, address indexed regulator);
    event PackageScanned(uint256 indexed batchId, uint256 scanCount, bool isSuspicious);

    modifier onlyRegulator() {
        require(msg.sender == regulator, "Access Denied: Only DRAP Regulator allowed");
        _;
    }

    modifier onlyRole(Role _role) {
        require(userRoles[msg.sender] == _role, "Access Denied: Unauthorized role");
        _;
    }

    constructor() {
        regulator = msg.sender;
    }

    function assignRole(address _user, Role _role) external onlyRegulator {
        userRoles[_user] = _role;
    }

    function registerBatch(uint256 _batchId, string memory _medicineName) external
onlyRole(Role.Manufacturer) {
        require(batches[_batchId].batchId == 0, "Batch ID already exists");
        batches[_batchId] = Batch(_batchId, _medicineName, msg.sender, Role.Manufacturer, false,
0);
        emit BatchRegistered(_batchId, _medicineName, msg.sender);
    }

    function transferCustody(uint256 _batchId, address _to, Role _nextStage) external {
        Batch storage b = batches[_batchId];
        require(b.batchId != 0, "Batch does not exist");
        require(b.currentOwner == msg.sender, "Only current owner can transfer custody");
        require(!b.isRecalled, "Transfer Blocked: Batch has been RECALLED!");
        b.currentOwner = _to;
        b.stage = _nextStage;
        emit CustodyTransferred(_batchId, msg.sender, _to, _nextStage);
    }

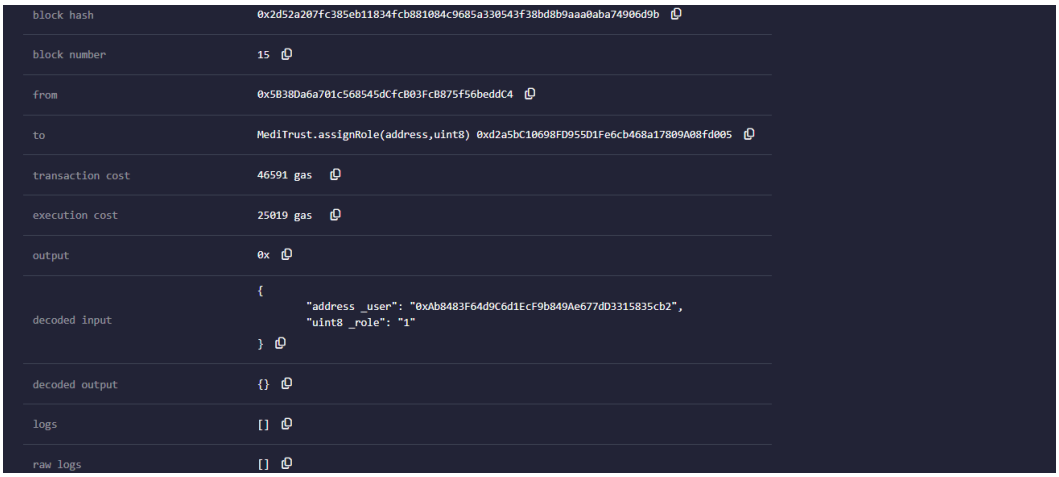
    function recallBatch(uint256 _batchId) external onlyRegulator {
        require(batches[_batchId].batchId != 0, "Batch does not exist");
        batches[_batchId].isRecalled = true;
        emit BatchRecalled(_batchId, msg.sender);
    }

    function verifyPackage(uint256 _batchId) external returns (
        string memory medicineName, address currentOwner, Role stage, bool isRecalled, uint256
scanCount, string memory alertMessage
    ) {
        Batch storage b = batches[_batchId];
        require(b.batchId != 0, "Error: Unregistered Package Code!");
        b.scanCount += 1;
        bool suspicious = b.scanCount > 1;
```

```
        emit PackageScanned(_batchId, b.scanCount, suspicious);
        string memory statusNote = "STATUS OK: Genuine Product";
        if (b.isRecalled) { statusNote = "DANGER: This batch has been RECALLED by DRAP!"; }
        else if (suspicious) { statusNote = "WARNING: Code scanned multiple times! Possible
Counterfeit Copy."; }
        return (b.medicineName, b.currentOwner, b.stage, b.isRecalled, b.scanCount, statusNote);
    }
}
```

4. Detailed Step-by-Step Validation & Execution Logs

- **Step 1: Deployment & Role Authorization:** Regulator account (0x5B38...) deployed MediTrust.sol and invoked assignRole(0xAb84..., 1). Gas consumed: 26,691 gas. Account 0xAb84... was successfully assigned Role.Manufacturer in global state.



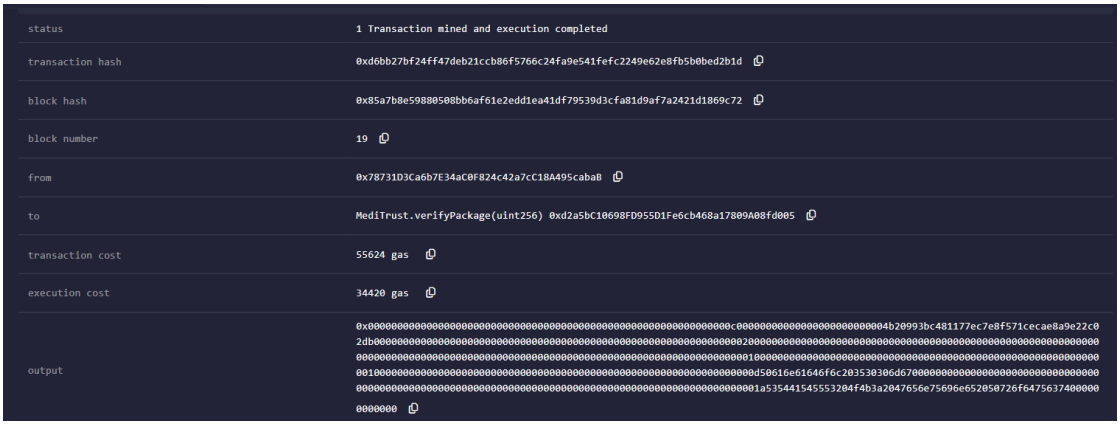
- **Step 2: Batch Registration:** Manufacturer (0xAb84...) registered Batch 101 with name 'Panadol 500mg'. Gas consumed: 98,300 gas. The BatchRegistered event was emitted with batchId = 101.

status	1 Transaction mined and execution completed
transaction hash	0x1e7130cea735287fbb2d2b278a8c2d2a7365e490538c41183bdb6733e992e2e1
block hash	0x02a78a2cfbd1552865446ca6c51bb0d51ce02f15399bd846d0b044fee3abd9d5
block number	17
from	0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
to	MediTrust.registerBatch(uint256,string) 0xd2a5bC10698FD955D1Fe6cb468a17809A08fd005
transaction cost	98300 gas
execution cost	76532 gas
output	0x
decoded input	{ "uint256 _batchId": "101", "string _medicineName": "Panadol 500mg" }

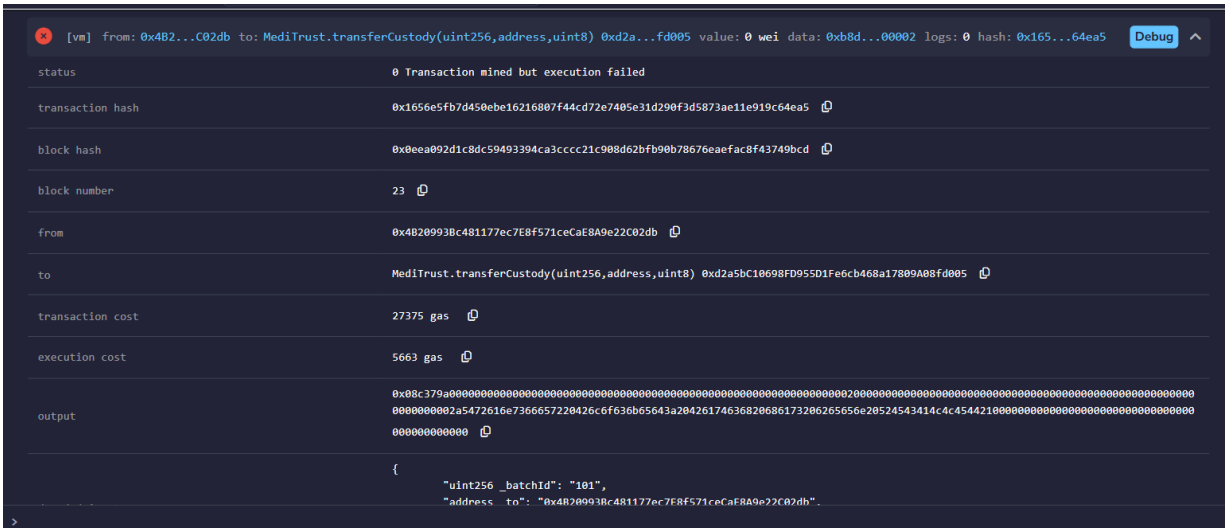
- **Step 3: Custody Handoff:** Manufacturer initiated custody transfer of Batch 101 to Distributor (0x4B20...) using stage = Role.Distributor. State updated currentOwner to 0x4B20... and stage to Distributor.

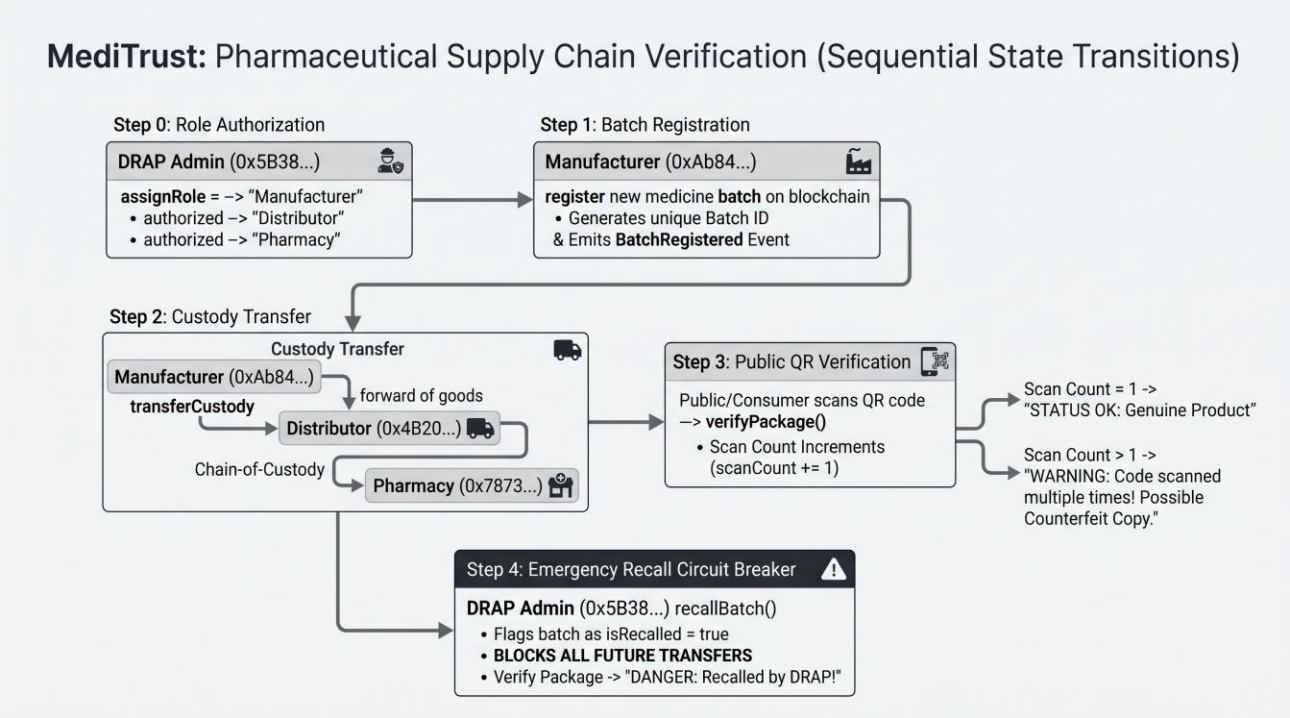
status	1 Transaction mined and execution completed
transaction hash	0xb506bae8731ac5a01e52ffcdc21d4944eaf67770f855d044511e18a3c5776aa4
block hash	0x3e228c6c845c965eac7551da4b2dd9348595da6a9ca05f4d20d4b39673cb57ea
block number	18
from	0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
to	MediTrust.transferCustody(uint256,address,uint8) 0xd2a5bC10698FD955D1Fe6cb468a17809A08fd005
transaction cost	32923 gas
execution cost	11211 gas
output	0x
decoded input	{ "uint256 _batchId": "101", "address _to": "0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db", "uint8 _nextStage": "2" }

- **Step 4: Public Verification & Counterfeit Alert Trigger:** First execution of verifyPackage(101) incremented scanCount to 1 and returned 'STATUS OK: Genuine Product'. Second execution incremented scanCount to 2, setting suspicious = true and returning 'WARNING: Code scanned multiple times! Possible Counterfeit Copy.'



- **Step 5: DRAP Emergency Recall Enforcement:** Regulator (0x5B38...) called recallBatch(101). Subsequent custody transfer attempts reverted with 'Transfer Blocked: Batch has been RECALLED!'. verifyPackage(101) updated output to 'DANGER: This batch has been RECALLED by DRAP!'





5. Summary & Phase 3 Roadmap

Phase 2 successfully verified smart contract deployment, role access control, state management, and real-time counterfeit/recall alerts in Remix IDE. Phase 3 will focus on compiling final presentation slides, capturing screenshots for the technical report, and preparing demonstration materials for evaluation.